

Rate Limiting Implementation

Overview

The application implements rate limiting to prevent API abuse and ensure fair usage across all users. The rate limiting is applied at the middleware level for all API endpoints.

Implementation Details

Rate Limits by Endpoint

- **/api/transcript-oxylabs**: 10 requests per minute
- **/api/format-transcript**: 20 requests per minute
- **Other API endpoints**: 30 requests per minute (default)

How It Works

1. **Client Identification**: Combines IP address and user agent to create unique identifiers
2. **In-Memory Storage**: Uses LRU cache to store request timestamps (suitable for single instances)
3. **Sliding Window**: Implements sliding window algorithm for accurate rate limiting
4. **Headers**: Returns standard rate limit headers with every response

Rate Limit Headers

```
X-RateLimit-Limit: 10      # Max requests allowed
X-RateLimit-Remaining: 7   # Requests remaining in window
X-RateLimit-Reset: 1234567 # Unix timestamp when limit resets
Retry-After: 45            # Seconds until next request (only on 429)
```

Rate Limit Response (429)

```
{
  "error": "Too Many Requests",
  "message": "Rate limit exceeded. Please try again later.",
  "retryAfter": 1234567890
}
```

Configuration

Edit `lib/rate-limiter.ts` to adjust rate limits:

```
export const RATE_LIMITS: Record<string, RateLimitConfig> = {
  '/api/your-endpoint': {
    interval: 60 * 1000, // Time window (ms)
```

```
    uniqueTokenPerInterval: 10, // Max requests  
  },  
};
```

Limitations

- **In-Memory Storage:** Rate limits are stored in memory per instance. In multi-region deployments, each region maintains its own rate limit counters.
- **No Persistence:** Rate limits reset when the server restarts.

Production Considerations

For production environments with multiple instances, consider:

1. **Vercel KV:** Use Vercel's KV storage for distributed rate limiting
2. **Redis:** Implement Redis-based rate limiting for consistent limits across regions
3. **Edge Config:** Use Vercel Edge Config for dynamic rate limit adjustments

Upgrading to Distributed Rate Limiting

To implement distributed rate limiting with Vercel KV:

1. Install Vercel KV: `npm install @vercel/kv`
2. Create KV store in Vercel dashboard
3. Update `rate-limiter.ts` to use KV instead of LRU cache
4. Deploy and test across regions